



CMPT 413/713: Natural Language Processing

Post-training: Reasoning

Spring 2026
2026-03-02

Slides adapted from Anoop Sarkar

Training stages

Piles of
unlabeled
text!



Self-supervised training
LM objective

$$p(w_t | w_{<t})$$

Pre-training

LM training on large, large
amount of data

Pre-training can be
broken into stages (mid-
training)

Text with
“instructions”
and
“responses”

List three fruit
Apple, orange,
banana

Supervised training
LM objective

$$p(w_t | w_{<t}; \text{prompt})$$

Instruction-tuning

Supervised fine-tuning for
instructions

Human
preference data
Data about what
people prefer



Reinforcement
learning

Preference optimization

Align to human
preferences

Post-training

(Can have more iterations)

Training for reasoning

Training LLMs to think step-by-step

- Supervised training with reasoning data and reasoning traces
- Reinforcement learning with verifiable rewards (RLVR)

Olmo training phases

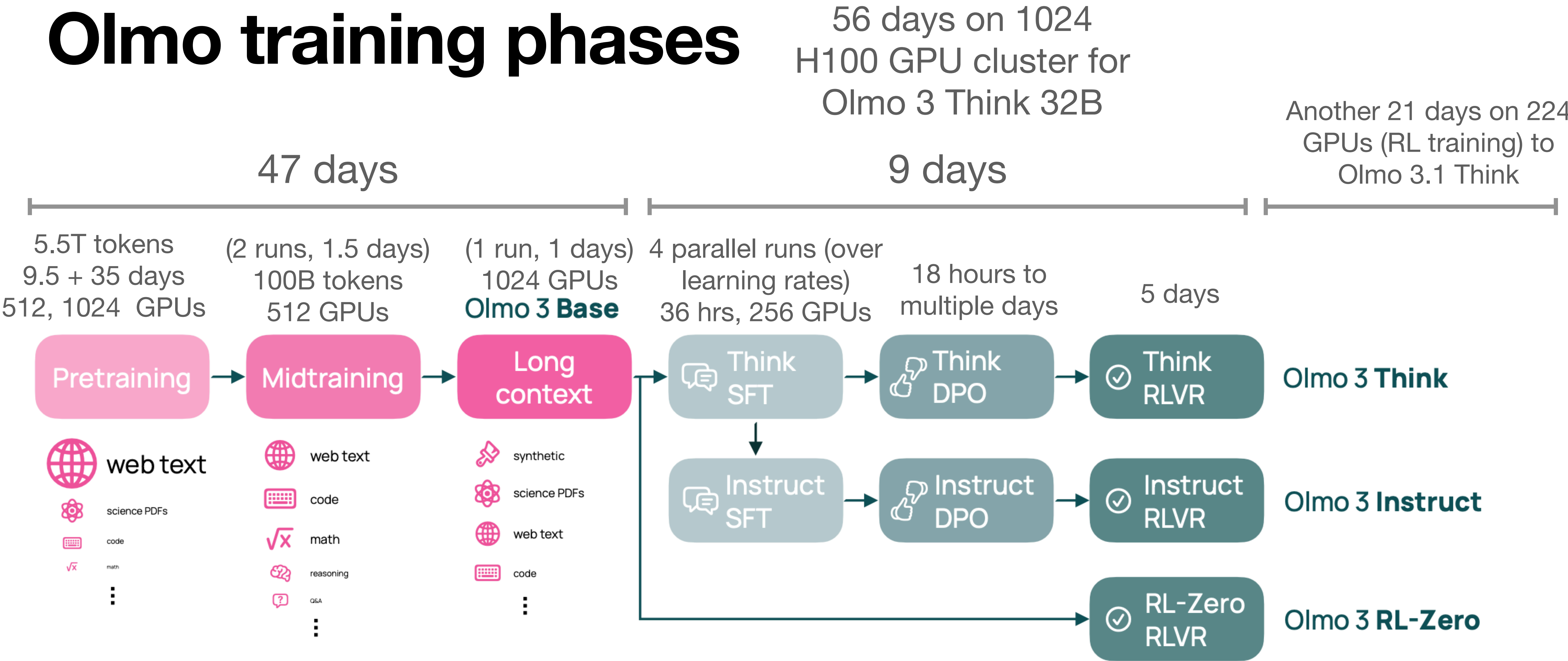
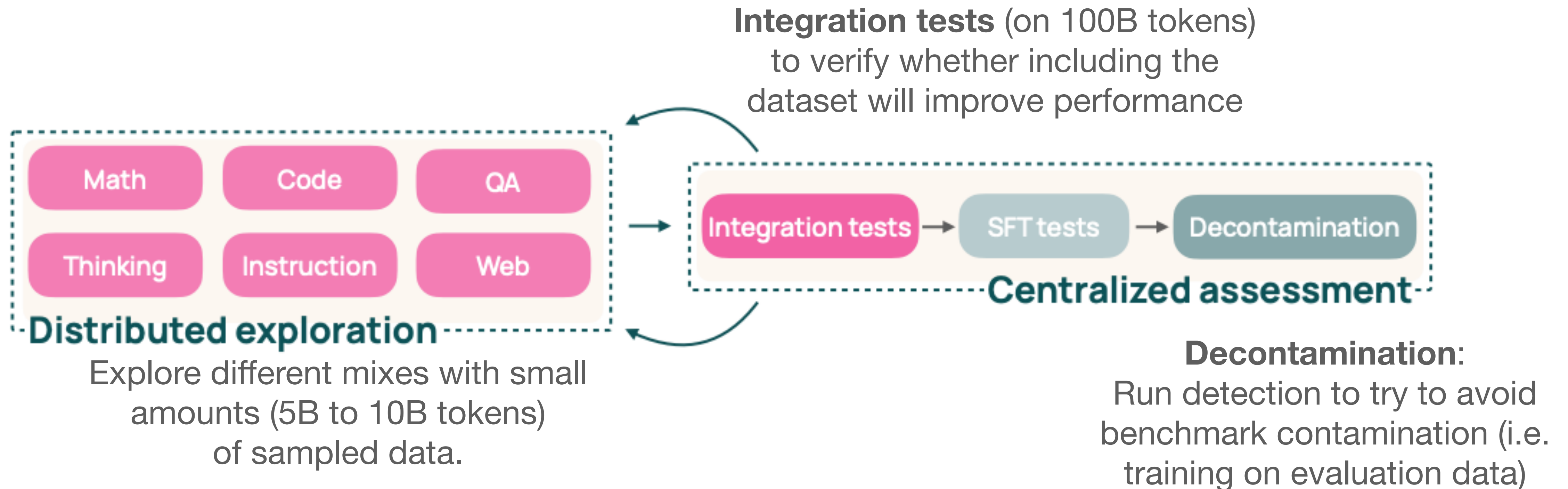


Figure 2 Depiction of model flow for Olmo 3. Development is divided into major **base model training (left)** and **post-training (right)** stages, each further divided into sub-stages with their own recipes (i.e., training data and method).

Figure from <https://arxiv.org/pdf/2512.13961> [Olmo 3, 2025]

Mid-training

Training on cleaned, specialized data



Olmo Post-training

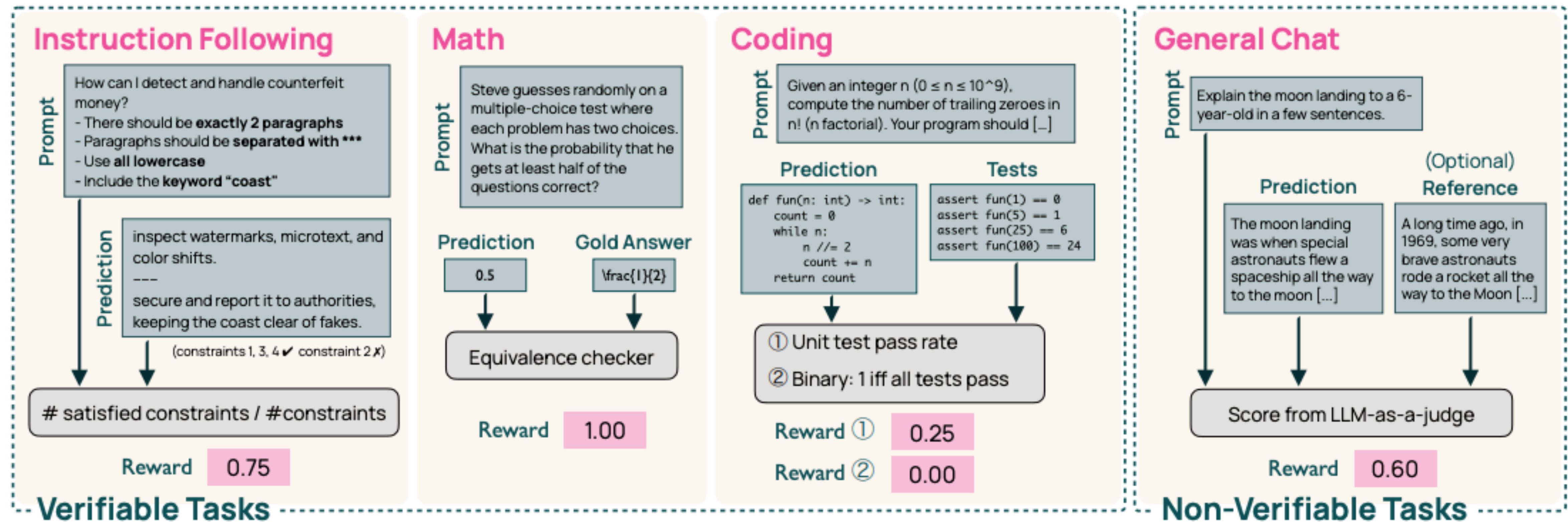
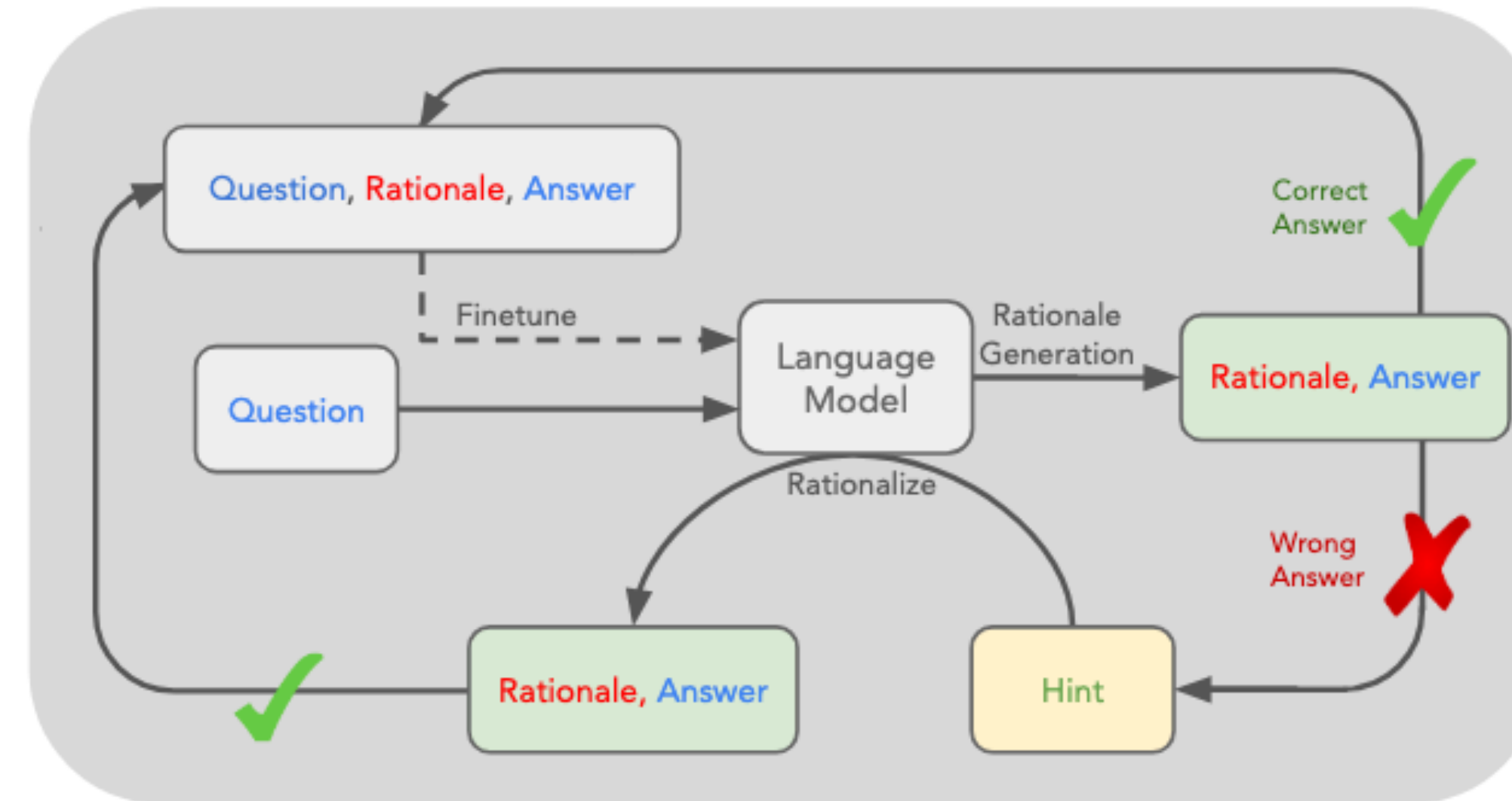


Figure 16 Verifiers and reward design for verifiable and non-verifiable tasks.

STaR: Self-Taught Reasoner

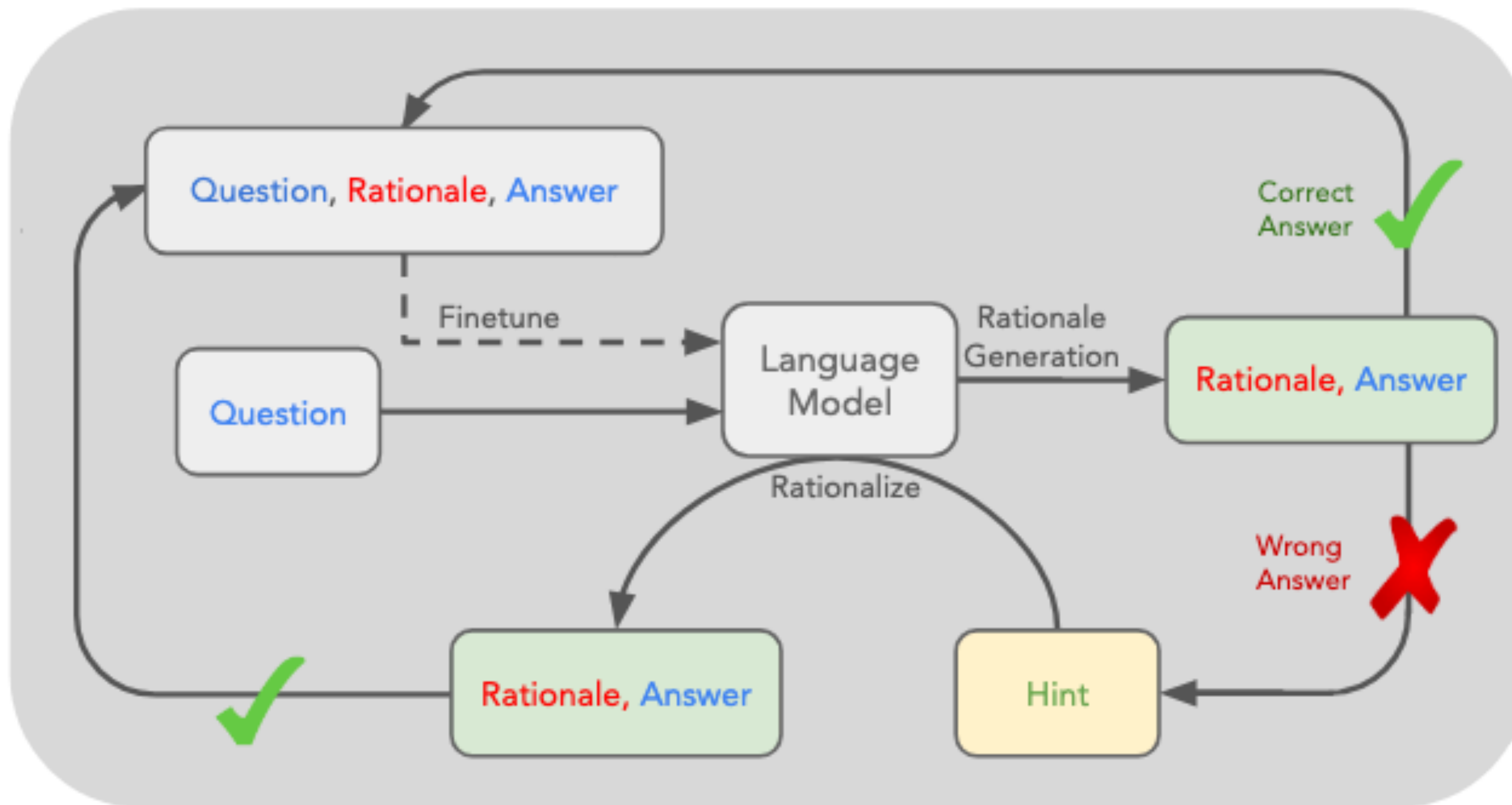
- Start with a pretrained LLM
- Assumes dataset with **questions** and **answers**.
- Generate **rationales**
- Fine-tune LLM on **correctly** generated answer with rationale
- If it cannot generate the correct answer, provide **hint** of the correct answer



Q: What can be used to carry a small dog?
Answer Choices:
(a) swimming pool
(b) basket
(c) dog show
(d) backyard
(e) own home
A: The answer must be something that can be used to carry a small dog. Baskets are designed to hold things. Therefore, the answer is basket (b).

Q: Where do you put your grapes just before checking out?
Answer Choices:
(a) mouth
(b) grocery cart (CORRECT)
(c) super market
(d) fruit basket
(e) fruit market

STaR: Self-Taught Reasoner



Q: What can be used to carry a small dog?
Answer Choices:
(a) swimming pool
(b) basket
(c) dog show
(d) backyard
(e) own home
A: The answer must be something that can be used to carry a small dog. Baskets are designed to hold things. Therefore, the answer is basket (b).

STaR: Self-Taught Reasoner

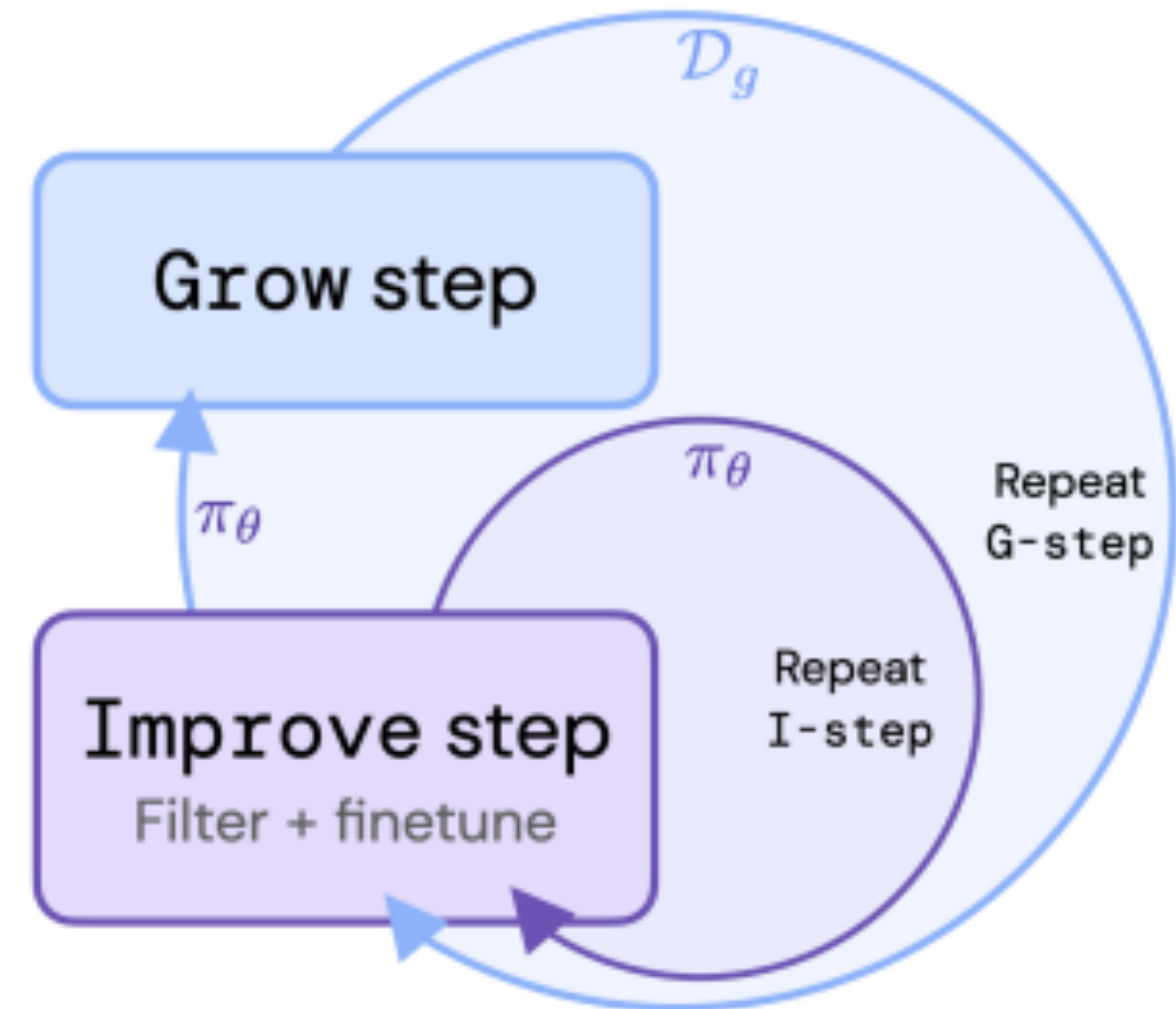
Algorithm 1 STaR

Input M : a pretrained LLM; dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^D$ (w/ few-shot prompts)

- 1: $M_0 \leftarrow M$ # Copy the original model
 - 2: **for** n **in** $1 \dots N$ **do** # Outer loop
 - 3: $(\hat{r}_i, \hat{y}_i) \leftarrow M_{n-1}(x_i) \quad \forall i \in [1, D]$ # Perform rationale generation
 - 4: $(\hat{r}_i^{\text{rat}}, \hat{y}_i^{\text{rat}}) \leftarrow M_{n-1}(\text{add_hint}(x_i, y_i)) \quad \forall i \in [1, D]$ # Perform rationalization
 - 5: $\mathcal{D}_n \leftarrow \{(x_i, \hat{r}_i, y_i) \mid i \in [1, D] \wedge \hat{y}_i = y_i\}$ # Filter rationales using ground truth answers
 - 6: $\mathcal{D}_n^{\text{rat}} \leftarrow \{(x_i, \hat{r}_i^{\text{rat}}, y_i) \mid i \in [1, D] \wedge \hat{y}_i \neq y_i \wedge \hat{y}_i^{\text{rat}} = y_i\}$ # Filter rationalized rationales
 - 7: $M_n \leftarrow \text{train}(M, \mathcal{D}_n \cup \mathcal{D}_n^{\text{rat}})$ # Finetune the original model on correct solutions - inner loop
 - 8: **end for**
-

ReST: Reinforced Self-Training

- Grow - generate dataset for training
- Improve - use generated dataset to fine-tune the policy



ReST: Reinforced Self-Training

Algorithm 1: ReST algorithm. *ReST* is a growing-batch RL algorithm. Given an initial policy of reasonable quality (for example, pre-trained using BC) iteratively applies Grow and Improve steps to update the policy. Here F is a filtering function, and $\mathcal{L}$ is an loss function.

Input: $\mathcal{D}$: Dataset, $\mathcal{D}_{eval}$: Evaluation dataset, $\mathcal{L}(\mathbf{x}, \mathbf{y}; \theta)$: loss, $R(\mathbf{x}, \mathbf{y})$: reward model, G : number of grow steps, I : number of improve steps, N : number of samples per context

Train π_θ on $\mathcal{D}$ using loss $\mathcal{L}$.

for $g = 1$ to G **do**

// Grow

 Generate dataset $\mathcal{D}_g$ by sampling: $\mathcal{D}_g = \{ (\mathbf{x}^i, \mathbf{y}^i) |_{i=1}^{N_g} \text{ s.t. } \mathbf{x}^i \sim \mathcal{D}, \mathbf{y}^i \sim \pi_\theta(\mathbf{y}|\mathbf{x}^i) \} \cup \mathcal{D}$.

 Annotate $\mathcal{D}_g$ with the reward model $R(\mathbf{x}, \mathbf{y})$.

for $i = 1$ to I **do**

// Improve

 Choose threshold s.t. $\tau_1 > V_{\pi_\theta}$ for $V_{\pi_\theta} = \mathbb{E}_{\mathcal{D}_g} [R(\mathbf{x}, \mathbf{y})]$ and $\tau_{i+1} > \tau_i$.

while *reward improves on $\mathcal{D}_{eval}$* **do**

 | Optimise θ on objective: $J(\theta) = \mathbb{E}_{(\mathbf{x}, \mathbf{y}) \sim \mathcal{D}_g} [F(\mathbf{x}, \mathbf{y}; \tau_i) \mathcal{L}(\mathbf{x}, \mathbf{y}; \theta)]$

end

end

end

Output: Policy π_θ

ReSTEM: EM for Reinforced Self-Training

- Expectation maximization for reinforced self-training - simpler version of ReST
 - Generate (E-step) - Generate data by sampling + annotate with reward
 - Improve (M-step) - Optimize parameters with generated data
- Differences from ReST
 - Focus on problems with binary rewards (0/1) vs bounded real-value rewards

ReSTEM: EM for Reinforced Self-Training

Algorithm 1: ReST (Expectation-Maximization). Given a initial policy (e.g., pre-trained LM), ReST^{EM} iteratively applies Generate and Improve steps to update the policy.

Input: $\mathcal{D}$: Training dataset, $\mathcal{D}_{val}$: Validation dataset, $\mathcal{L}(\mathbf{x}, \mathbf{y}; \theta)$: loss, $r(\mathbf{x}, \mathbf{y})$: Non-negative reward function, I : number of iterations, N : number of samples per context

for $i = 1$ **to** I **do**

// Generate (E-step)

 Generate dataset $\mathcal{D}_i$ by sampling: $\mathcal{D}_i = \{ (\mathbf{x}^j, \mathbf{y}^j) |_{j=1}^N \text{ s.t. } \mathbf{x}^j \sim \mathcal{D}, \mathbf{y}^j \sim p_\theta(\mathbf{y}|\mathbf{x}^j) \}$

 Annotate $\mathcal{D}_i$ with the reward $r(\mathbf{x}, \mathbf{y})$.

// Improve (M-step)

while *reward improves on $\mathcal{D}_{val}$* **do**

 | Optimise θ to maximize objective: $J(\theta) = \mathbb{E}_{(\mathbf{x}, \mathbf{y}) \sim \mathcal{D}_i} [r(\mathbf{x}, \mathbf{y}) \log p_\theta(\mathbf{y}|\mathbf{x})]$

end

end

Output: Policy p_θ

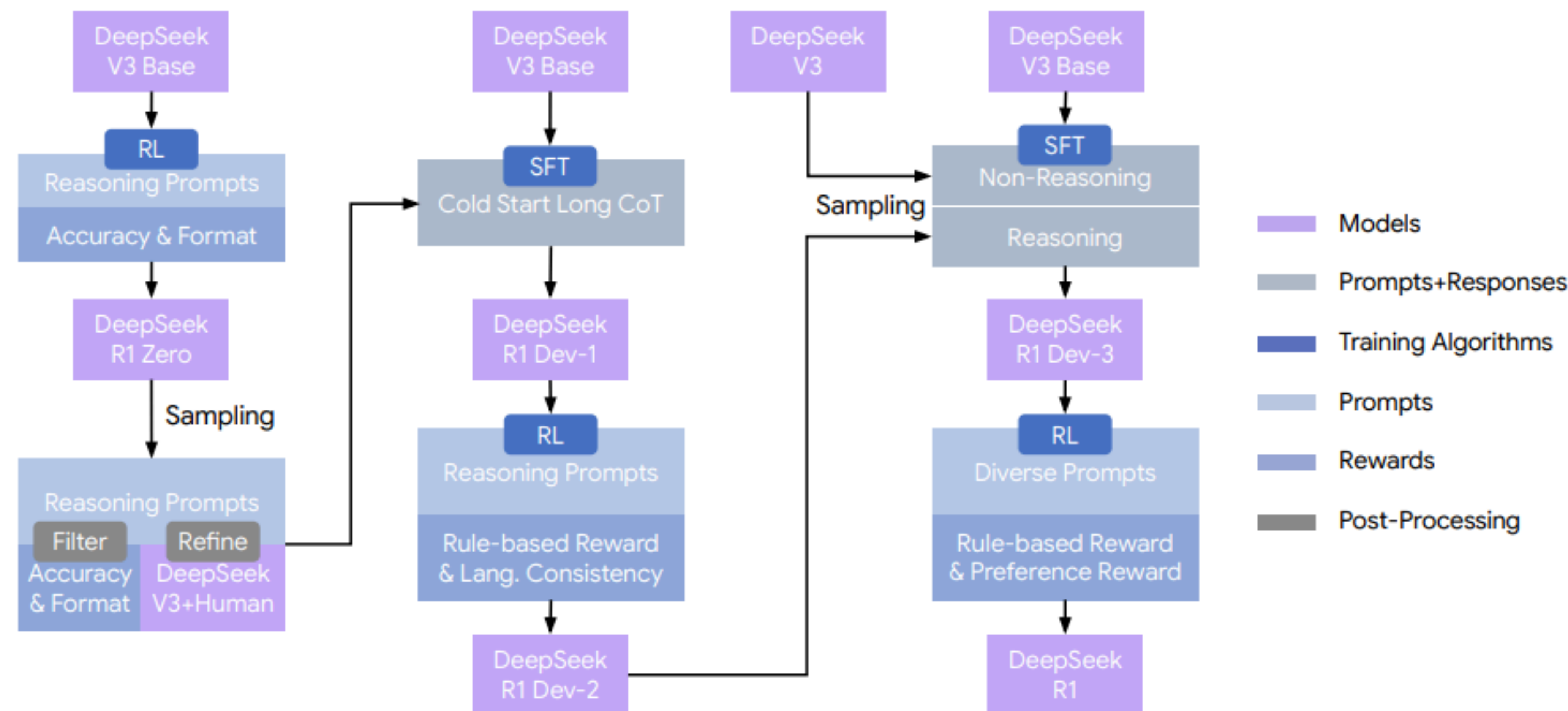
ReSTEM: EM for Reinforced Self-Training

	ReSTEM	ReST	STaR	RFT
Starts from fine-tuned model	✗	✓	✗	✗
Finetunes from base model in each iteration	✓	✗	✓	N/A
Uses rationalizations for unsolved questions	✗	✗	✓	✗
Temperature sampling for exploration	✓	✓	✗	✓
Experiments with Large LMs	✓	✗	✗	✓
Multiple iterations	✓	✓	✓	✗
Larger gains on bigger models	✓	N/A	N/A	✗
Evaluation on held out tasks	✓	✗	✗	✗

Deepseek R1

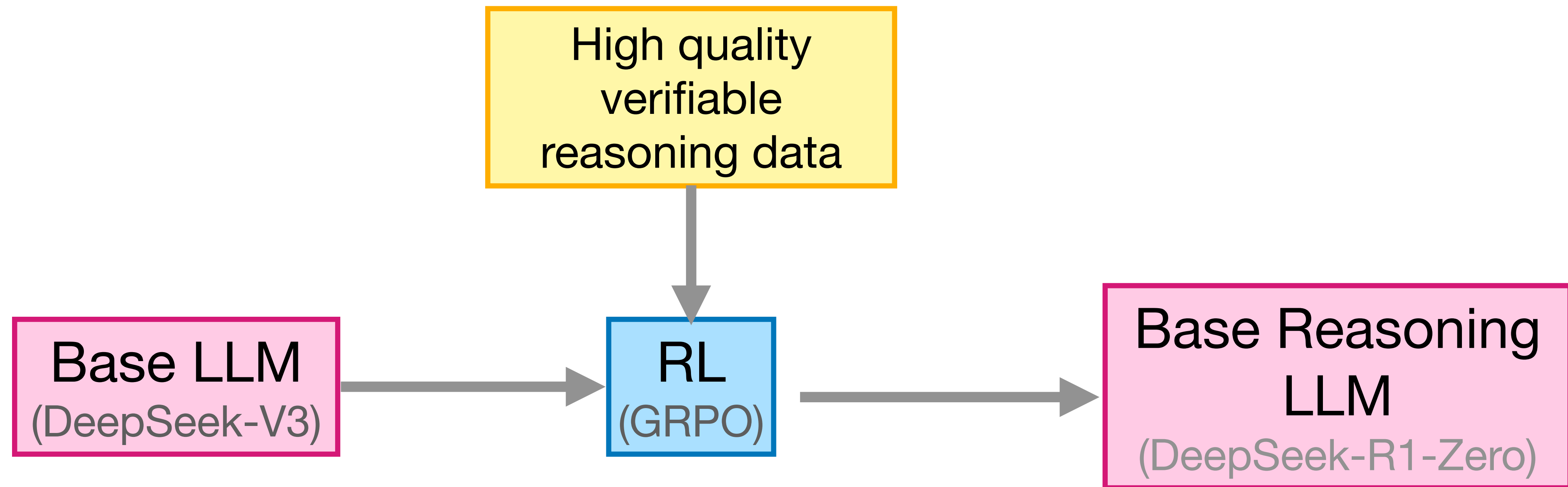
- DeepSeek-V3 - MoE model trained with multi-token prediction
- DeepSeek-R1-Zero - Use RL to improve base model (DeepSeek-V3) for reasoning (e.g. generate “thought” process)
- DeepSeek-R1 - Improved version with cold-start, supervised fine-tuning on generated / filtered data.

Deepseek R1



Deepseek R1-Zero

Training recipe



DeepSeek R1-Zero

Training recipe

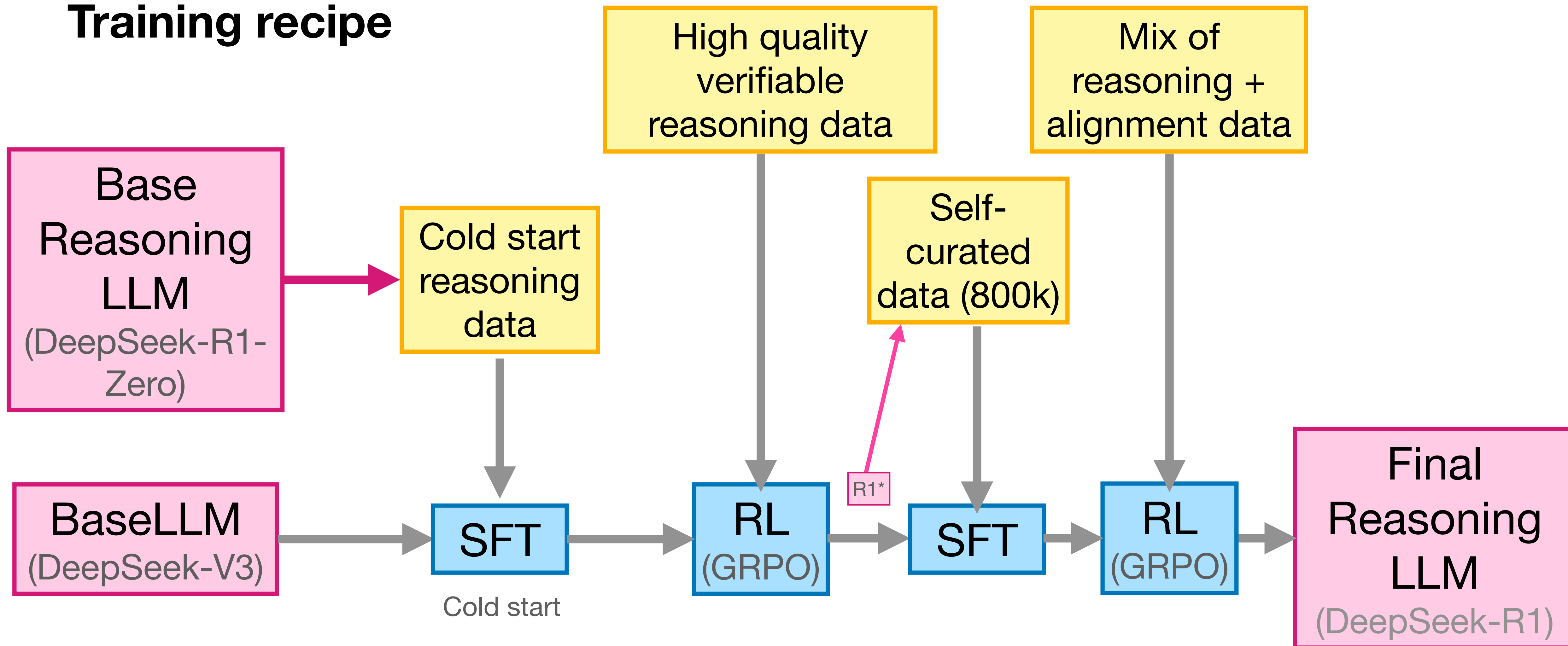
- Train LLM to explicitly generate reasoning process

A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. The reasoning process and answer are enclosed within `<think>` `</think>` and `<answer>` `</answer>` tags, respectively, i.e., `<think> reasoning process here </think>` `<answer> answer here </answer>`. User: **prompt**. Assistant:

- **Accuracy reward** for whether response is correct and **format reward** to provide details of thinking process
- Additional **language consistency reward** is added when training DeepSeek-R1

DeepSeek R1

Training recipe



Reinforcement Learning Review

Some slides adapted from Graham Neubig and others

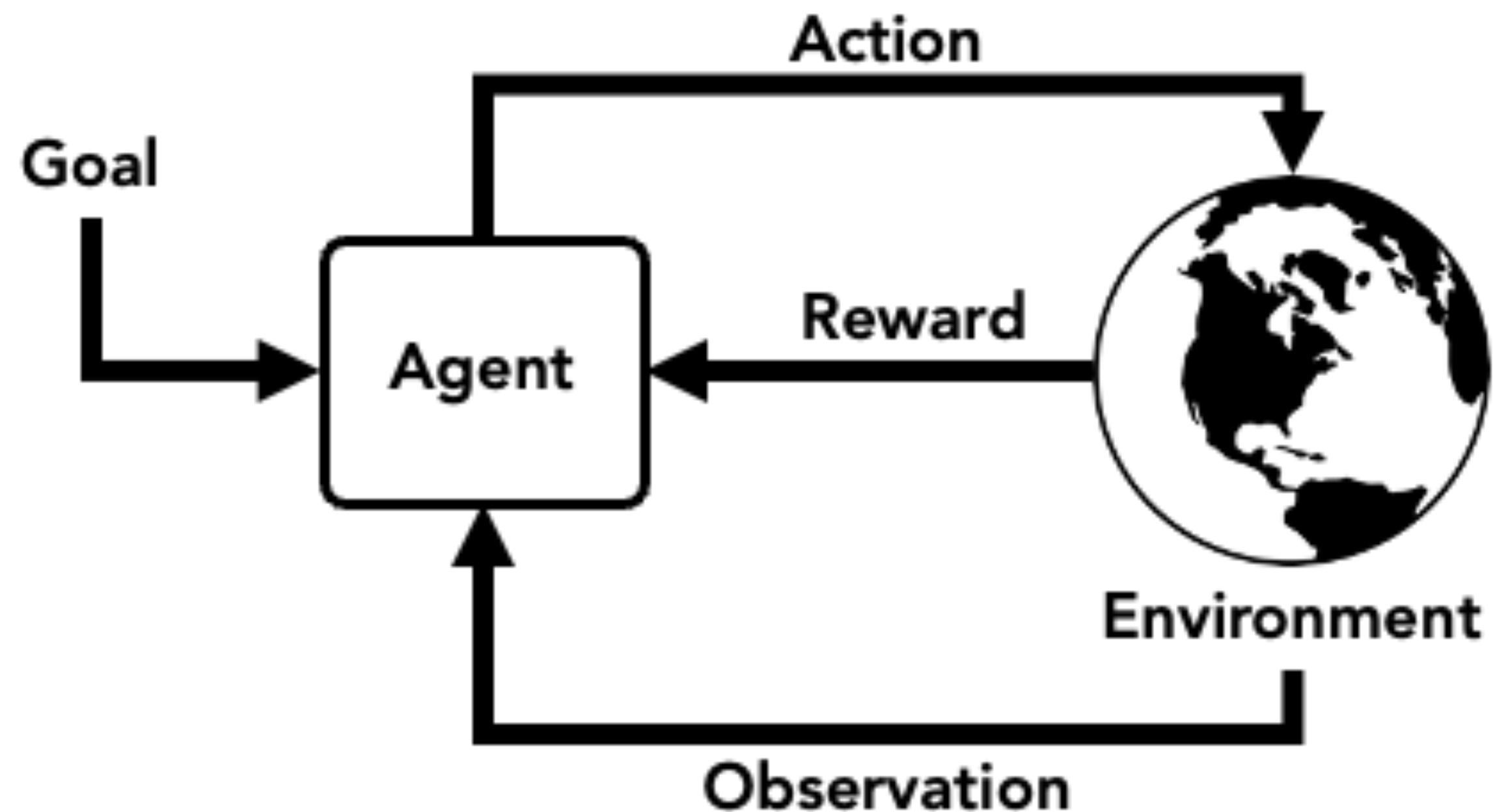
What is reinforcement learning?

Learning where we have an

- Environment X
 - Ability to make actions A
 - Get delayed reward R
- Environment provides feedback
 - No examples of optimal policy

For text generation, we use RL when there is a **sequence-level evaluation metric** that is difficult to optimize without first generating the whole sentence.

Can also be used for longer dialog where reward is at the end (e.g. customer achieved their goal).



Reinforcement learning

Determine policy to **maximize expected accumulated reward**.

Typically modelled as POMDP (sequence of states with partial observations)

- **Actions:** What token to output?
- **Policy:** What action(s) to take given sequence of observations and actions?
 - Policy models the probability of action given state
 - For text generation, what sequence of tokens y to generate given input x
tokens: $\pi(a, s) = P(y | x)$
- **Reward:**
 - RLHF: Provided by reward model trained on human preference
 - RLVR: Provided by verifiable reward / reward

Reward in RL

- For an action at time step t , state s_t and action a_t , get a reward $r_t = R(s_t, a_t)$
- Goal is to maximize **total reward** over time $\sum_{t=0}^{\infty} r_t$
- But maybe better to get immediate reward rather than wait. So maximize **discounted reward** $\sum_{t=0}^{\infty} \gamma^t r_t$
- Everything is stochastic
 - **expected discounted reward** for a policy π

$$\mathbb{E}_{\pi} \left[\sum_t^{\infty} \gamma^t r_t \right]$$

Reward in RL

- For a given trajectory $\tau = ((s_0, a_0), (s_1, a_1), \dots)$, which is a sequence of states and actions in the world, we can have the following rewards.

Also known as **return**

- Total reward:** $R_{\text{total}}(\tau) = \sum_{t=0}^{\infty} r_t$

- Discounted reward:** $R_{\text{discounted}}(\tau) = \sum_{t=0}^{\infty} \gamma^t r_t$

Algorithms that optimize parameters of the policy directly are called **policy-based methods**

- Find a policy π to maximize **expected discounted reward**:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} = \int_{\tau} P(\tau | \pi) R(\tau) = \left[\sum_t^{\infty} \gamma^t r_t \right]$$

RL for text generation

- **State:** input text ($X = x_1, x_2, \dots$) and text that is generated so far ($y_1, y_2, \dots, y_{t-1}$)
- **Action:** next token to generate y_t
- **Trajectory of actions** (generated text): $Y = y_1, \dots, y_n$
- **Objective:** Maximize $J_{\text{RL}}(X) = R(Y) \log P(Y|X)$
- **Loss:**

$$\ell_{\text{RL}}(X) = -R(Y) \log P(Y|X)$$

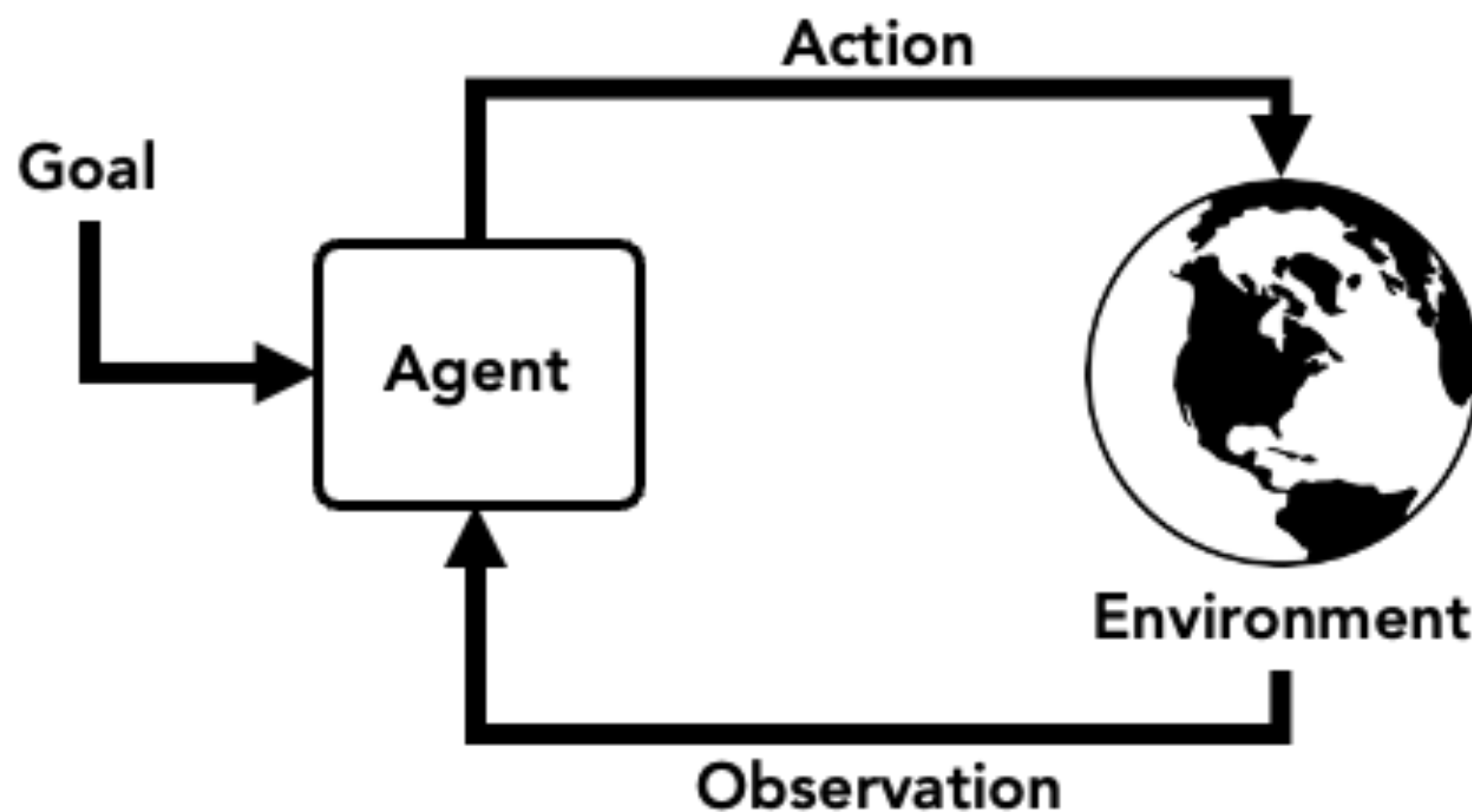
How is this related to supervised learning?

Supervised learning

In normal supervised learning, we do maximum likelihood estimation (MLE)

In the RL literature, this is also known as **imitation learning** where there is an “expert” demonstrating what actions to take at each time step

- Have expert demonstrations (possibly interactive)



$$\ell_{\text{MLE}}(Y|X) = -\log P(Y|X)$$

Training set with
Input text X and output text Y

Examples of imitation learning algorithms:

- Behavior cloning - collect data (expert demonstrations once) and then train
- DAgger - iteratively collect data (as it encounters new states)

Self-training

- **Self-training:** If we start with a trained model, we can sample (or take argmax) to get data points (e.g. “demonstrations”) for training.

$$\hat{Y} \sim P(Y|X) \quad \text{or} \quad \hat{Y} = \arg \max_Y P(Y|X)$$

$$\ell_{\text{self}}(X) = -\log P(\hat{Y}|X)$$

Policy gradient / REINFORCE

- **Self-training:** If we start with a trained model, we can sample (or take argmax) to get data points (e.g. “demonstrations”) for training.

$$\hat{Y} \sim P(Y|X) \quad \text{or} \quad \hat{Y} = \arg \max_Y P(Y|X)$$

$$\ell_{\text{self}}(X) = -\log P(\hat{Y}|X)$$

- **RL:** Add term to scale loss by the reward

$$\ell_{\text{REINFORCE}}(X) = -R(\hat{Y}) \log P(\hat{Y}|X)$$

Optimize policy using stochastic gradient descent

- **But some issues:**
 - step too small, training too slow,
 - too large, training is unstable (too much variability)

Regularization to existing model

- **KL-regularization:** TRPO (trust region policy optimization)

Improve reward

Keep model similar

$$\ell_{\text{KL-reg}} = \rho(\hat{Y}, X) R(\hat{Y}) - \beta \text{KL}[P(\cdot|X; \theta_{\text{old}}), P(\cdot|X; \theta)]$$

$$\rho(Y, X) = \frac{P(Y|X; \theta)}{P(Y|X; \theta_{\text{old}})} \quad \text{Ratio function}$$

- **Add regularization term to keep model similar to initial model**
- **Tricky to implement**

Regularization to existing model

- **KL-regularization:**

Improve reward

Keep model similar

$$\ell_{\text{KL-reg}} = \rho(\hat{Y}, X) R(\hat{Y}) - \beta \text{KL}[P(\cdot|X; \theta_{\text{old}}), P(\cdot|X; \theta)]$$

$$\rho(Y, X) = \frac{P(Y|X; \theta)}{P(Y|X; \theta_{\text{old}})}$$

Ratio function

- **Proximal policy optimization (PPO)**

Prevent jumps

$$\ell_{\text{PPO}} = \min \left(\rho(\hat{Y}, X) R(\hat{Y}), \text{clip}(\rho(\hat{Y}, X), 1 + \epsilon, 1 - \epsilon) R(\hat{Y}) \right)$$

Going beyond reward

Hard to compute, so use
neural networks to estimate
these functions
(critic training)

- State **Value** Function
 - How good is the state s_t ?
- State-Action Value Function (**Q function**)
 - How good is a particular action a at state s_t ?
- **Advantage**
 - How much better is taking action a than the average?

$$V_{\pi}(s_t) = \mathbb{E}_{\pi} \left[\sum_{i=t}^{\infty} \gamma^{i-t} r_i \right]$$

$$Q_{\pi}(s_t, a) = \mathbb{E}_{s_{t+1}} \left[R(s_t, a, s_{t+1}) + \gamma [V_{\pi}(s_{t+1})] \right]$$

$$A_{\pi}(s_t, a) = Q_{\pi}(s_t, a) - V_{\pi}(s_t)$$

Use as return function
in optimization:

$$\ell(X) = -R(\hat{Y}) \log P(\hat{Y}|X) \longrightarrow \ell(X) = -A(\hat{Y}) \log P(\hat{Y}|X)$$

PPO with advantage

- **Proximal policy optimization (PPO)**

Prevent jumps

$$\ell_{\text{PPO}} = \min \left(\rho(\hat{Y}, X) \underline{\hat{A}(\hat{Y})}, \text{clip}(\rho(\hat{Y}, X), 1 + \epsilon, 1 - \epsilon) \underline{\hat{A}(\hat{Y})} \right)$$

$$\rho(Y, X) = \frac{P(Y|X; \theta)}{P(Y|X; \theta_{\text{old}})} \quad \text{Ratio function}$$

On-policy, actor-critic algorithm

Actor: Policy network - determines what action to take

Critic: Value network - evaluates action by estimating the value function
(so we can estimate the advantage function) $A(\hat{Y})$

RLHF with general policy gradient method

- Initialize the policy π_{ϕ}^{RL} to π^{SFT} , the policy output from SFT

- Loop for many steps

- Initialize a new empty dataset $D_{\pi_{\phi}^{\text{RL}}}$

- Loop for many steps

- Sample a random prompt x from D_{RL}

- Generate a response y from the policy π_{ϕ}^{RL}

- Calculate the reward signal $r_{\theta}(x, y)$ from r_{θ}

- Add the triple $(x, y, r_{\theta}(x, y))$ to $D_{\pi_{\phi}^{\text{RL}}}$
- $$\text{objective}(\phi) = E_{(x,y) \sim D_{\pi_{\phi}^{\text{RL}}}} \left[r_{\theta}(x, y) - \beta \log \left(\frac{\pi_{\phi}^{\text{RL}}(y|x)}{\pi^{\text{SFT}}(y|x)} \right) \right]$$

- Use policy gradient method to increase objective:

RLHF with PPO

Algorithm 1 PPO

- 1: Input: initial policy parameters θ_0 , initial value function parameters ϕ_0 .
- 2: **for** $n = 0, 1, 2, \dots$ **do**
- 3: Collect a set of trajectories $\mathcal{D}_n = \{\tau_i\}$ by executing policy $\pi(\theta_n)$ within the environment.
- 4: Compute rewards-to-go $\hat{R}_t$.
- 5: Compute advantage estimates, $\hat{A}_t$ (using any advantage estimation method) based on the current value function V_{ϕ_n} .
- 6: Update the policy by maximizing the PPO-penalty/clip/ptx objective:

$$\theta_{n+1} = \arg \max_{\theta} \mathcal{L}_{\text{ppo-clip}}(\theta_n).$$

- 7: Update the value function by regression on mean-squared error:

$$\phi_{n+1} = \arg \min_{\phi} \mathcal{L}_{\text{critic}}(\phi_n).$$

- 8: **end for**
-

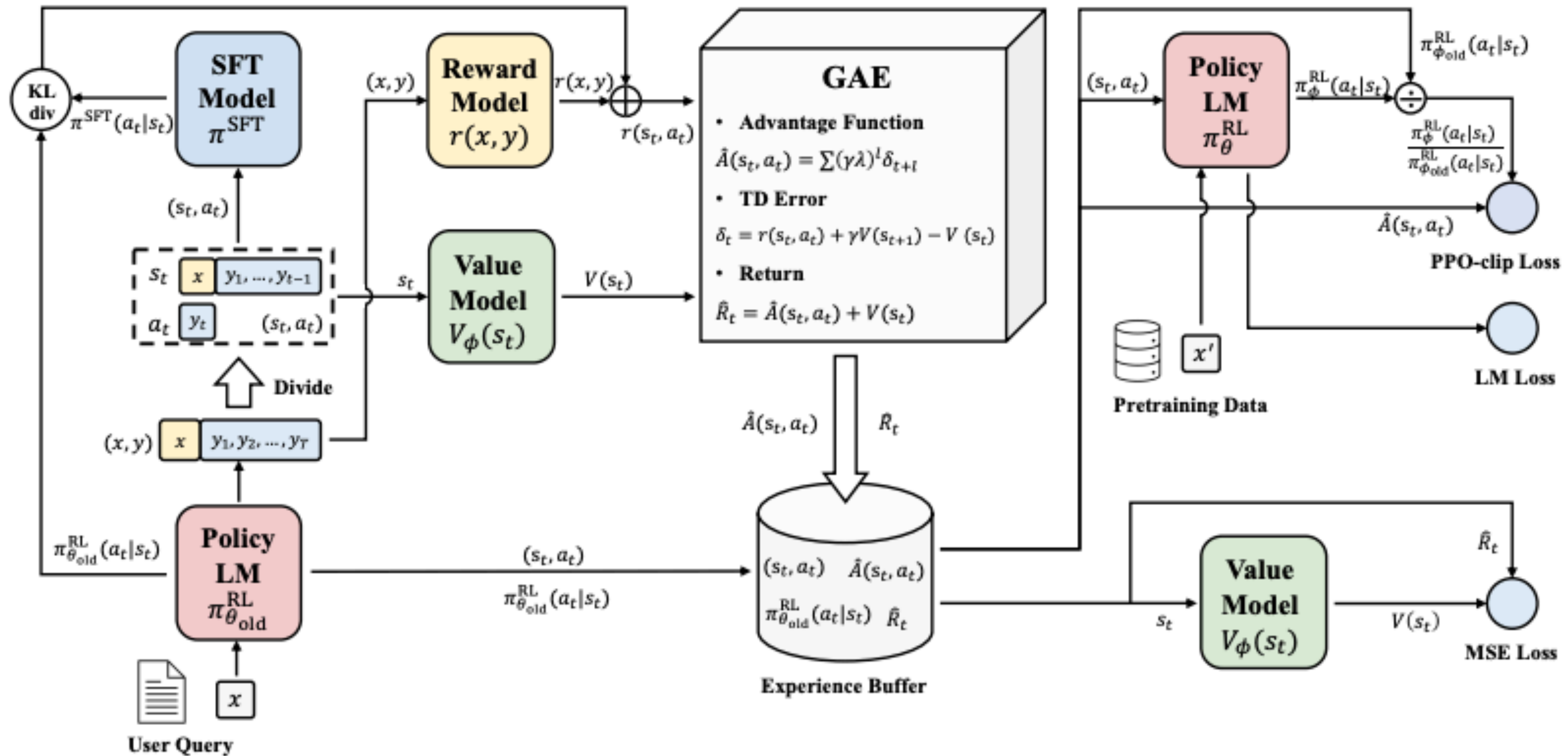


Figure 1: PPO workflow, depicting the sequential steps in the algorithm's execution. The process begins with sampling from the environment, followed by the application of GAE for improved advantage approximation. The diagram then illustrates the computation of various loss functions employed in PPO, signifying the iterative nature of the learning process and the policy updates derived from these losses.

Group Relative Policy Optimization (GRPO)

- Replace advantage function estimated with neural reward model with advantage function that is computed from rule-based reward model
- Compute advantage as how much newer policy improves over older policy

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL} [\pi_{\theta} || \pi_{ref}] \right\}.$$

$$\mathbb{D}_{KL} (\pi_{\theta} || \pi_{ref}) = \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - \log \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - 1,$$

Group Relative Policy Optimization (GRPO)

- Outcome supervision

$$\hat{A}_{i,t} = \tilde{r}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$$

$$\mathbf{r} = \{r_1, \dots, r_G\}$$

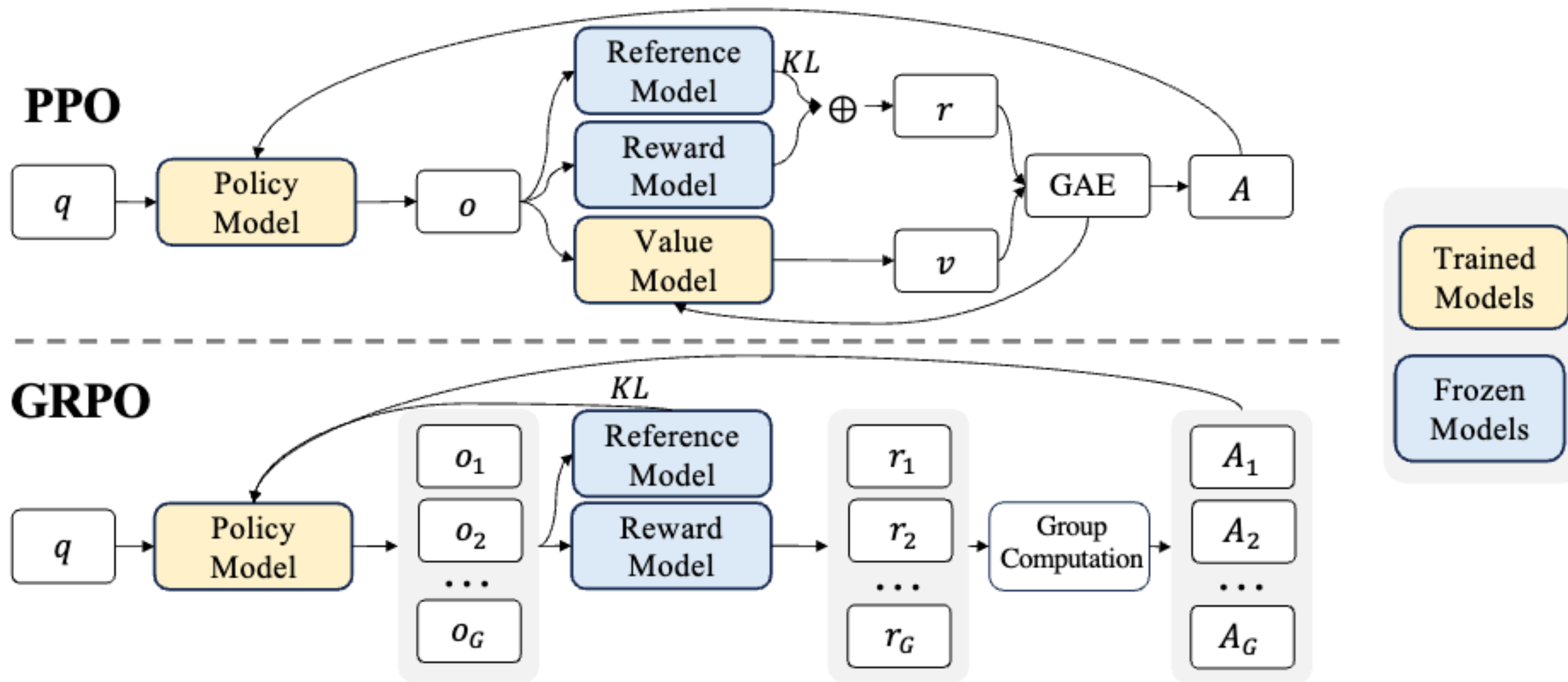
- Process supervision

$$\hat{A}_{i,t} = \sum_{\text{index}(j) \geq t} \tilde{r}_i^{\text{index}(j)}$$

$$\tilde{r}_i^{\text{index}(j)} = \frac{r_i^{\text{index}(j)} - \text{mean}(\mathbf{R})}{\text{std}(\mathbf{R})}$$

$$\mathbf{r} = \{ \{ r_1^{\text{index}(1)}, \dots, r_1^{\text{index}(K_1)} \}, \dots, \{ r_G^{\text{index}(1)}, \dots, r_G^{\text{index}(K_G)} \} \}$$

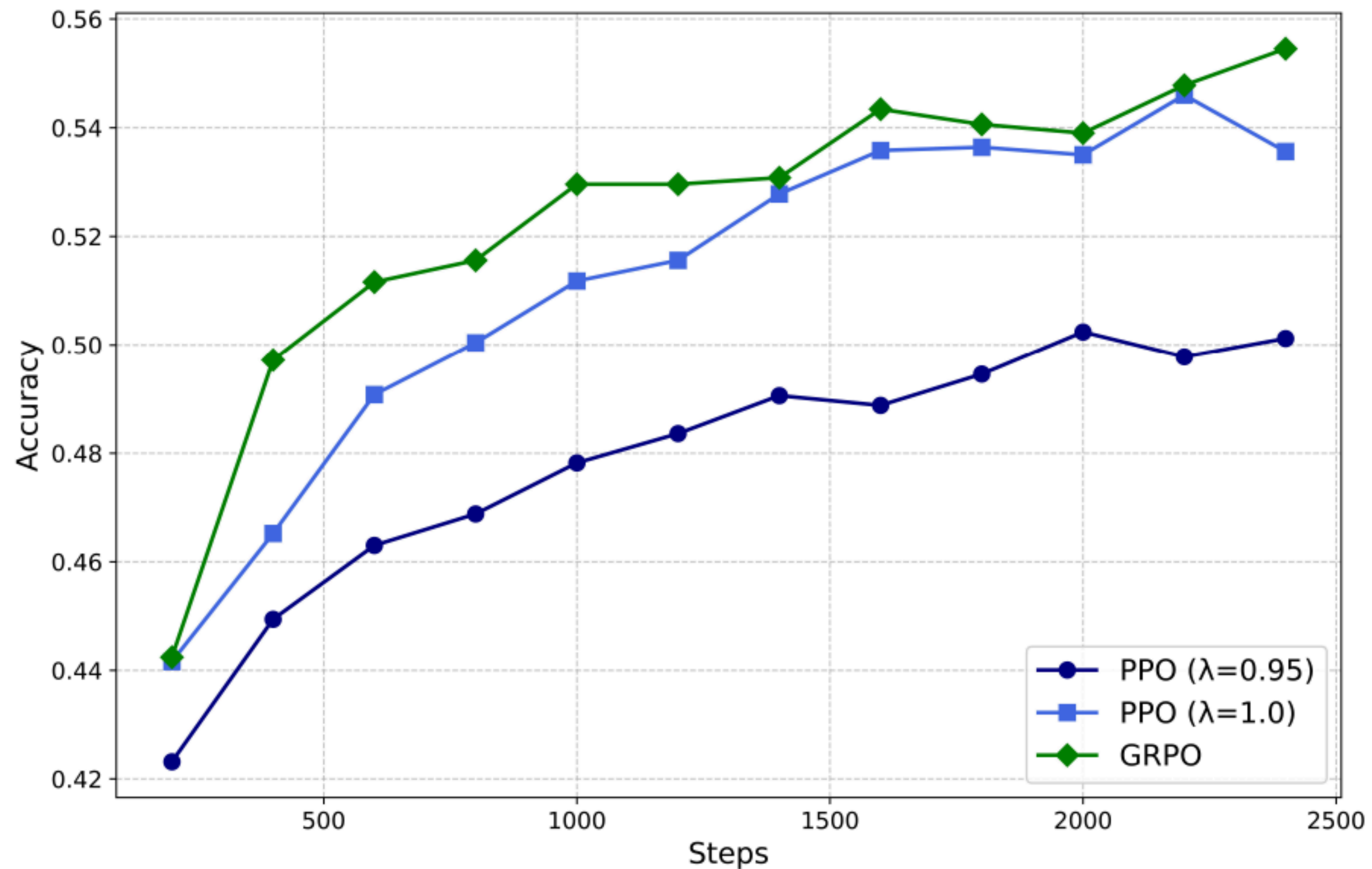
Group Relative Policy Optimization (GRPO)



Eliminate value model

Comparison of PPO and GRPO

MATH task



DeepSeek-R1 performance

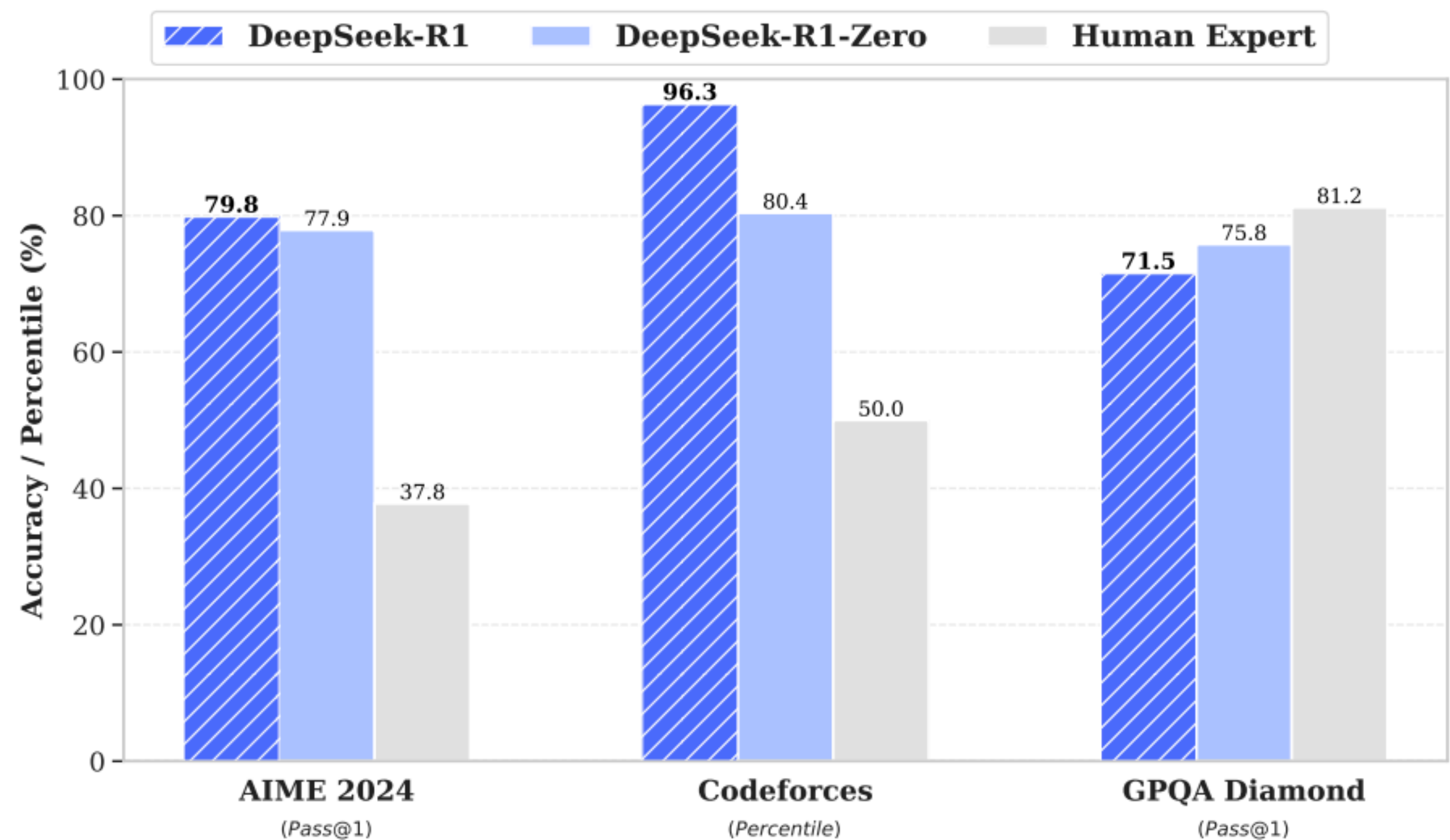
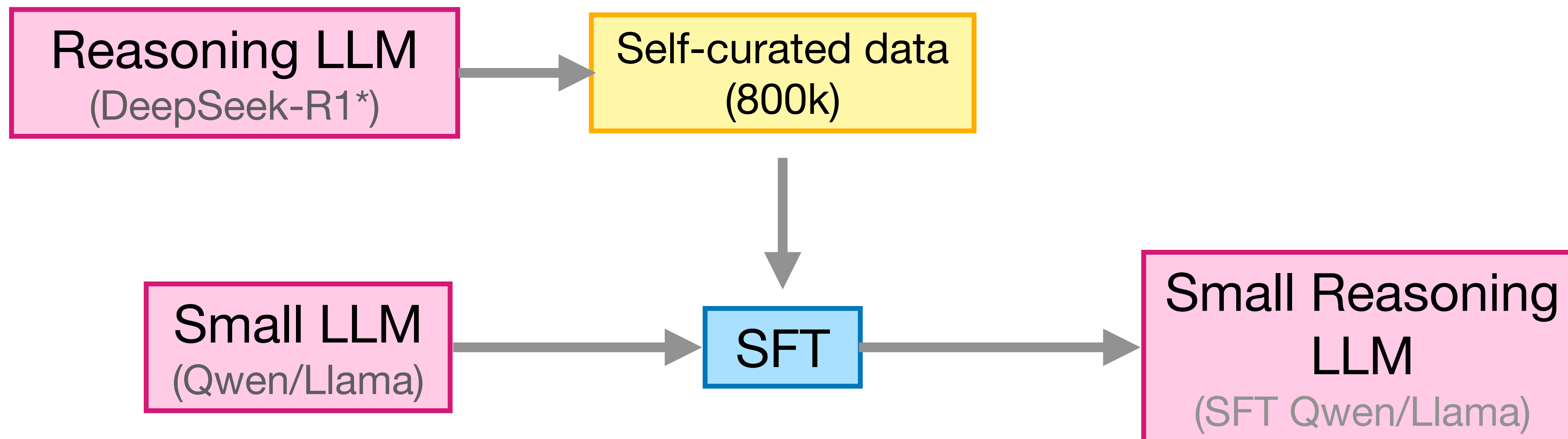


Figure 10 | The benchmark performance of DeepSeek-R1 and DeepSeek-R1-Zero is compared with human scores across different datasets. For AIME and Codeforces, the human scores represent the average performance of all human competitors. In the case of GPQA, the human score corresponds to Ph.D.-level individuals who had access to the web for answering the questions.

Distillation of DeepSeek-R1

Training recipe

Generated / Filtered using DeepSeek-R1*
Reasoning (600K)
General (200K)



DeepSeek-R1 performance

Distilled models

Table 15 | Comparison of DeepSeek-R1 distilled models and other comparable models on reasoning-related benchmarks. Numbers in bold denote the performance is statistically significant (t-test with $p < 0.01$).

Model	AIME 2024		MATH	GPQA	LiveCode	CodeForces
	pass@1	cons@64		Diamond	Bench	
	pass@1	cons@64	pass@1	pass@1	pass@1	rating
GPT-4o-0513	9.3	13.4	74.6	49.9	32.9	759
Claude-3.5-Sonnet-1022	16.0	26.7	78.3	65.0	38.9	717
DeepSeek-R1-Distill-Qwen-1.5B	28.9	52.7	83.9	33.8	16.9	954
DeepSeek-R1-Distill-Qwen-7B	55.5	83.3	92.8	49.1	37.6	1189
DeepSeek-R1-Distill-Qwen-14B	69.7	80.0	93.9	59.1	53.1	1481
DeepSeek-R1-Distill-Qwen-32B	72.6	83.3	94.3	62.1	57.2	1691
DeepSeek-R1-Distill-Llama-8B	50.4	80.0	89.1	49.0	39.6	1205
DeepSeek-R1-Distill-Llama-70B	70.0	86.7	94.5	65.2	57.5	1633

Does RL increase the reasoning capacity?

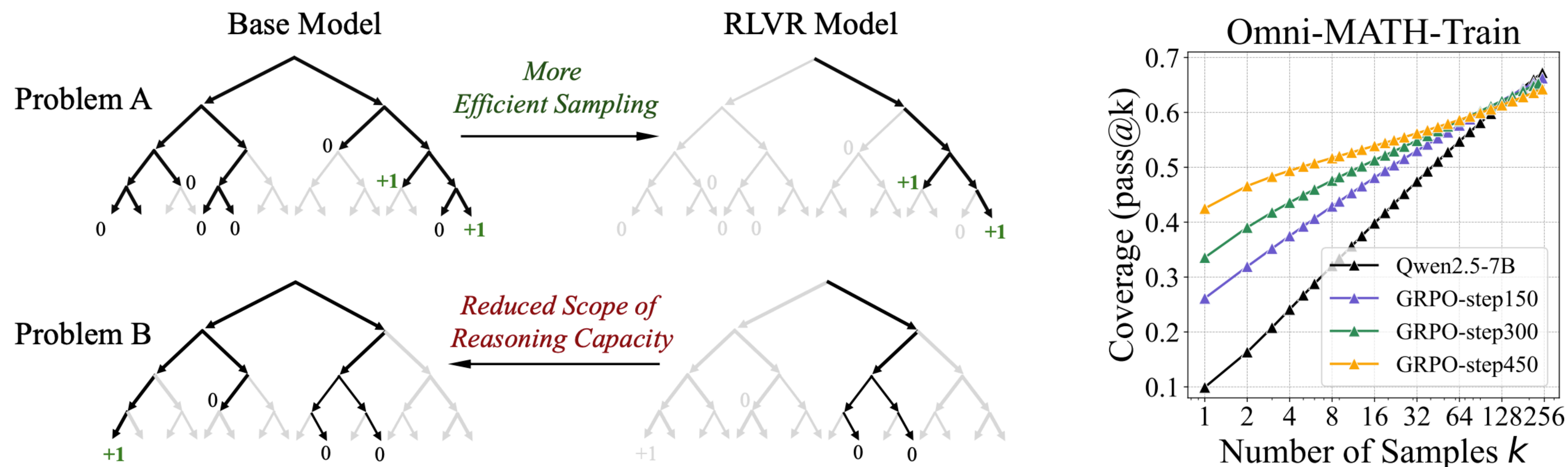


Figure 1: **(Left)** The effect of current RLVR on LLM’s reasoning ability. Search trees are generated by repeated sampling from the base and RLVR-trained models for a given problem. Grey indicates paths that are unlikely to be sampled by the model, while **black** indicates paths that are likely to be sampled. **Green** indicates correct paths, which has positive rewards. Our key finding is that all reasoning paths in the RLVR model are already present in the base model. For certain problems like Problem A, RLVR training biases the distribution toward rewarded paths, improving sampling efficiency. However, this comes at the cost of reduced scope of reasoning capacity: For other problems like Problem B, the base model contains the correct path, whereas that of the RLVR model does not. **(Right)** As RLVR training progresses, the average performance (*i.e.*, pass@1) improves, but the coverage of solvable problems (*i.e.*, pass@256) decreases, indicating a reduction in LLM’s reasoning boundary.

